

Image Recognition



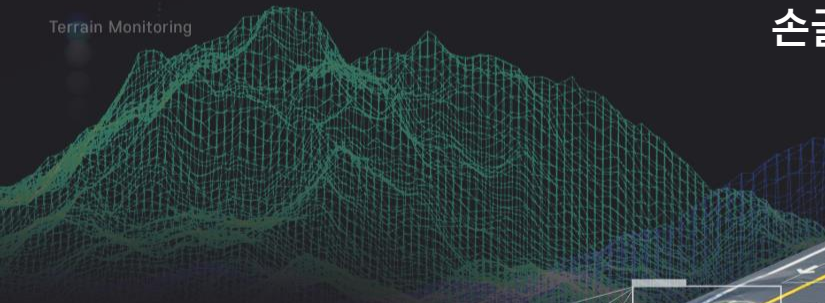
강원혁신플랫폼

컴퓨터 비전



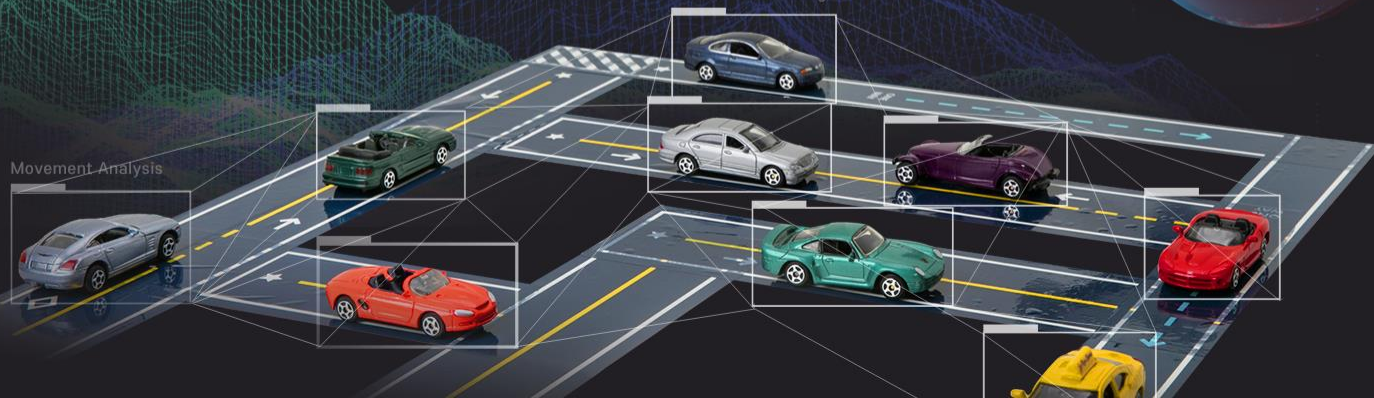
손글씨 숫자 분류 학습

Terrain Monitoring



Autonomous Driving

Movement Analysis



MNIST 데이터셋을 이용하여 0~9까지의 숫자를 분류하는
Multi Classification 모델을 만들고 이 모델을 이용해
우리가 직접 손으로 작성한 숫자 이미지를 분류 예측한 결과를 확인해보자

이것은 다중 이미지 분류(Image Multi Classification)의 문제임

MNIST 손글씨 숫자 데이터를 이용한
10가지 이미지 분류 전체 과정 실습



인공신경망(딥러닝)의
세부 알고리즘 학습

10가지 클래스 인덱스

“0”:0 “1”:1 “2”:2 “3”:3 “4”:4
“5”:5 “6”:6 “7”:7 “8”:8 “9”:9

※ 실습은 구글 Colab을 이용함



MNIST 데이터셋을 이용하여 **0~9**까지의 숫자 분류 데이터를 이용해 학습 수행하기

- ▶ 구글 Colab 노트북 화면에서 실행 버튼 클릭
- ▶ 실행 결과 TensorFlow 버전이 '2.12.0'인 것을 알 수 있음



```
1  # tensorflow 버전 확인
2
3  import tensorflow as tf
4
5  tf.__version__
```

📄 '2.12.0'

MNIST 데이터셋을 이용하여 0~9까지의 숫자 분류 데이터를 이용해 학습 수행하기

- ▶ 내 구글 드라이브에 연동
- ▶ 구글 Colab 노트북 화면에서 실행 버튼 클릭



```
# 내 구글 드라이브에 연동  
from google.colab import drive
```

```
drive.mount('/content/gdrive')
```



```
Mounted at /content/gdrive
```


MNIST 데이터셋을 이용하여 0~9까지의 숫자 분류 데이터를 이용해 학습 수행하기

- ➡ [Google Drive에 연결] > [계정 선택] > [구글 드라이버 접근 허용] 메뉴 선택
- ➡ Google Drive에 연동되면 구글 드라이브에 저장된 데이터를 활용할 수 있음

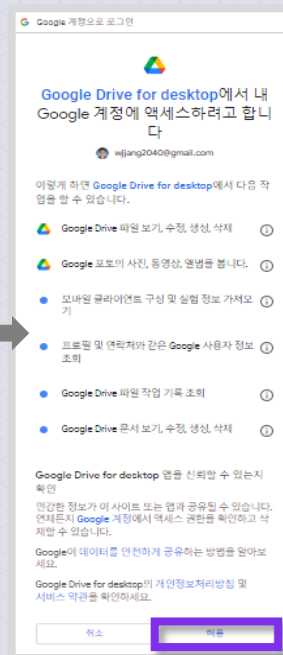
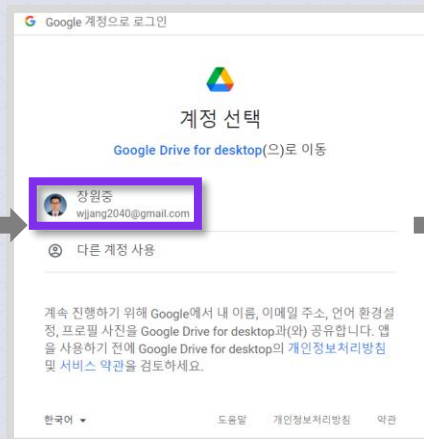
참고 아래와 같이 Google Drive에 연동

노트북에서 Google Drive 파일에 액세스하도록 허용하시겠습니까?

이 노트북에서 Google Drive 파일에 대한 액세스를 요청합니다. Google Drive에 대한 액세스 권한을 부여하면 노트북에서 실행되는 코드가 Google Drive의 파일을 수정할 수 있게 됩니다. 이 액세스를 허용하기 전에 노트북 코드를 검토하시기 바랍니다.

아니요

Google Drive에 연결



MNIST 데이터셋을 이용하여 0~9까지의 숫자 분류 데이터를 이용해 학습 수행하기

- ➡ Magic command를 실행하기 전에 TensorFlow 패키지를 Import 하면 안됨
- ➡ 반드시 Magic command 먼저 실행

```
# tensorflow 2.x 버전으로 변경하는 magic command를 실행한다.
```

```
%tensorflow_version 2.x
```

반드시 먼저 실행

```
import tensorflow as tf
```

```
# 접속되어 있는 VM 이 GPU 를 사용하고 있는지 확인합니다.
```

```
device_name = tf.test.gpu_device_name()
```

```
print("device name:", device_name)
```

```
if device_name != '/device:GPU:0':
```

```
    raise SystemError('GPU device not found')
```

```
print('Found GPU at: {}'.format(device_name))
```

```
print(tf.__version__)
```



```
Colab only includes TensorFlow 2.x; %tensorflow_version has no effect.
```

```
device name: /device:GPU:0
```

```
Found GPU at: /device:GPU:0
```

```
2.8.2
```

MNIST 데이터셋을 이용하여 **0~9**까지의 숫자 분류 데이터를 이용해 **학습** 수행하기

🔄 구글 Colab 노트북 화면에서 **실행 버튼** 클릭



필요로 하는 패키지와 모듈을 불러옵니다.

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt
import math
```

MNIST 데이터셋을 이용하여 **0~9**까지의 숫자 분류 데이터를 이용해 학습 수행하기

- ➡ 구글 Colab 노트북 화면에서 실행 버튼 클릭
- ➡ 배치 크기는 128, 10개 클래스 분류, 반복학습은 100 에폭으로 설정



상수 정의

batch_size = 128

batch size

num_classes = 10

분류 클래스 종류(0 ~ 9)

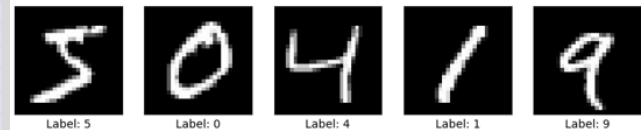
epochs = 100

training 횟수

MNIST 데이터셋을 이용하여 0~9까지의 숫자 분류 데이터를 이용해 학습 수행하기

- ➡ **show_sample()** 함수는 출력할 이미지와 레이블을 넘겨받아 25개 샘플 이미지를 화면에 출력하는 함수를 정의함

```
1 # 25개의 샘플 이미지를 Display 해 주는 함수 정의
2
3 def show_sample(images, labels, sample_count=25):
4
5     # Create a square with can fit {sample_count} images
6     grid_count = math.ceil(math.ceil(math.sqrt(sample_count)))
7     grid_count = min(grid_count, len(images), len(labels))
8
9     plt.figure(figsize=(2*grid_count, 2*grid_count))
10    for i in range(sample_count):
11        plt.subplot(grid_count, grid_count, i+1)
12        plt.xticks([])
13        plt.yticks([])
14        plt.grid(False)
15        plt.imshow(images[i], cmap=plt.cm.gray)
16        plt.xlabel(labels[i])
17    plt.show()
```



MNIST 데이터셋을 이용하여 0~9까지의 숫자 분류 데이터를 이용해 학습 수행하기

- ➡ `show_sample_digit()` 함수는 출력할 이미지, 레이블, 특정한 숫자를 넘겨받아 특정한 숫자(Digit)만 25개 이미지를 화면에 출력하는 함수를 정의함

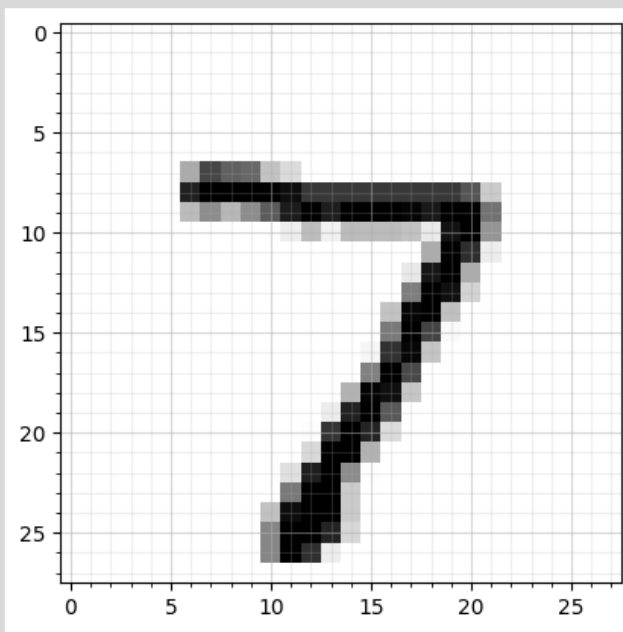
```
1 # 특정한 숫자(Digit)만 25개 Display 해 주는 함수 정의
2
3 def show_sample_digit(images, labels, digit, sample_count=25):
4
5     # Create a square with can fit {sample_count} images
6     grid_count = math.ceil(math.ceil(math.sqrt(sample_count)))
7     grid_count = min(grid_count, len(images), len(labels))
8
9     plt.figure(figsize=(2*grid_count, 2*grid_count))
10    i = 0
11    digit_count = 0
12    while digit_count < sample_count:
13        i += 1
14        if (digit == labels[i]):
15            plt.subplot(grid_count, grid_count, digit_count+1)
16            plt.xticks([])
17            plt.yticks([])
18            plt.grid(False)
19            plt.imshow(images[i], cmap=plt.cm.gray)
20            plt.xlabel(labels[i])
21            digit_count += 1
22    plt.show()
```



MNIST 데이터셋을 이용하여 0~9까지의 숫자 분류 데이터를 이용해 학습 수행하기

- ➡ show_digit_image() 특정한 숫자(Digit)의 이미지와 예측한 Label을 출력하는 함수를 정의함
- ➡ 구글 Colab 노트북 화면에서 실행 버튼 클릭

```
1 # 특정한 숫자(digit)의 이미지와 예측한 label 을 Display 해주는 함수 정의
2
3 def show_digit_image(image):
4     # Draw digit image
5     fig = plt.figure()
6     ax = fig.add_subplot(1, 1, 1)
7
8     # Major ticks every 20, minor ticks every 5
9     major_ticks = np.arange(0, 29, 5)
10    minor_ticks = np.arange(0, 29, 1)
11    ax.set_xticks(major_ticks)
12    ax.set_xticks(minor_ticks, minor=True)
13    ax.set_yticks(major_ticks)
14    ax.set_yticks(minor_ticks, minor=True)
15
16    # And a corresponding grid
17    ax.grid(which='both')
18
19    # Or if you want different settings for the grids:
20    ax.grid(which='minor', alpha=0.2)
21    ax.grid(which='major', alpha=0.5)
22    ax.imshow(image, cmap=plt.cm.binary)
23
24    plt.show()
```





■ MNIST 데이터셋 목적

- 🔄 이미지 분류 모델을 Training 시키고 Testing 하기 위해서 다운로드함

■ MNIST 데이터셋 구성

손으로 작성된
60,000개의 훈련용 이미지

+

10,000개의 테스트 이미지

- 🔄 MNIST 데이터셋에 있는 각각의 이미지들은 단 하나의 숫자(Digit)만 갖고 있음
- 🔄 28 × 28 그레이스케일(Grayscale) 이미지로 되어있음

MNIST 데이터셋을 이용하여 0~9까지의 숫자 분류 데이터를 이용해 학습 수행하기

- ➡ Keras에 있는 MNIST 데이터셋 다운로드
- ➡ 훈련용 데이터 60,000개와 테스트 데이터 10,000개가 있음

▶ # Keras에 있는 MNIST dataset을 다운로드 받습니다.

```
mnist = keras.datasets.mnist
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

print("Train Data :", train_images.shape, train_labels.shape)
print("Test Data :", test_images.shape, test_labels.shape)
```

↳ Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11493376/11490434 [=====] - 0s 0us/step
11501568/11490434 [=====] - 0s 0us/step
Train Data : (60000, 28, 28) (60000,)
Test Data : (10000, 28, 28) (10000,)

MNIST 데이터셋을 이용하여 0~9까지의 숫자 분류 데이터를 이용해 학습 수행하기

🔄 훈련용 데이터셋의 0번째 이미지 값을 출력한 결과, 0~255 사이의 값으로 구성됨



원본 이미지 출력

```
print(train_images[0, 0:10, 0:10])
```



```
[[ 0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0 30 36]
 [ 0  0  0  0  0  0  0 49 238 253]
 [ 0  0  0  0  0  0  0 18 219 253]
 [ 0  0  0  0  0  0  0  0 80 156]]
```



0

255

Grayscale Level Change

● **훈련용** 데이터셋의 **0번째 이미지 값**을 출력한 결과, **0~1 사이의 값**으로 구성됨

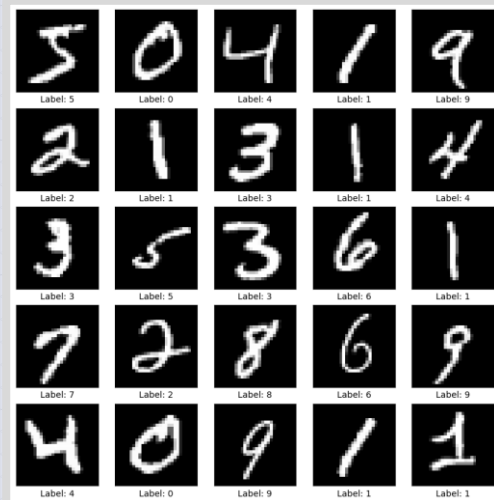
정규화 : $0 \sim 255 \rightarrow 0 \sim 1$

```
(60000, 28, 28) (10000, 28, 28)
[[0. 0. 0. 0. 0. 0.
  0. 0. 0. 0. 0. 0.
  [0. 0. 0. 0. 0. 0.
  0. 0. 0. 0. 0. 0.
  [0. 0. 0. 0. 0. 0.
  0. 0. 0. 0. 0. 0.
  [0. 0. 0. 0. 0. 0.
  0. 0. 0. 0. 0. 0.
  [0. 0. 0. 0. 0. 0.
  0. 0. 0. 0. 0. 0.
  [0. 0. 0. 0. 0. 0.
  0. 0. 0. 0. 0. 0.
  [0. 0. 0. 0. 0. 0.
  0. 0. 0.00046136 0.00055363]
  [0. 0. 0. 0. 0. 0.
  0. 0.00075356 0.00366013 0.00389081]
  [0. 0. 0. 0. 0. 0.
  0. 0.00027682 0.00336794 0.00389081]
  [0. 0. 0. 0. 0. 0.
  0. 0. 0.0012303 0.002399081]]
```

MNIST 데이터셋을 이용하여 0~9까지의 숫자 분류 데이터를 이용해 학습 수행하기

- ➡ show_sample() 함수로 훈련용 데이터셋에서 샘플 이미지 25개를 출력함
- ➡ 출력할 이미지와 레이블에 인자 값을 전달함

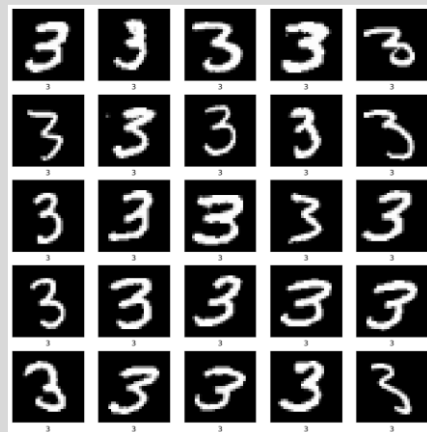
```
1 # training dataset 에서 25개의 샘플 이미지를 Display 해 줍니다.
2
3 show_sample(train_images, ['Label: %s' % label for label in train_labels])
```



MNIST 데이터셋을 이용하여 0~9까지의 숫자 분류 데이터를 이용해 학습 수행하기

- ➡ show_sample_digit() 함수로 훈련용 데이터셋에서 특정한 숫자(Digit)만 25개를 출력함
- ➡ 이미지, 레이블, 특정한 숫자(여기서는 숫자 3)을 넘겨받아 특정한 숫자 25개를 출력함

```
1 # show_sample_digit 함수로 training dataset에서 특정한 숫자(Digit)만 25개 Display 해 줍니다.
2 # - 이미지와 레이블을 이자값으로 전달
3
4 show_sample_digit(train_images, train_labels, 3)
```





■ Keras를 이용해서 이미지 분류를 위한 인공신경망 네트워크 구성하기

- ② CNN, Fully Connected Neural Network, Relu, Max Pooling, Dropout 알고리즘 이용
- ② 아래 코드와 같이 4층 인공신경망 네트워크 모델 생성



인공신경망을 구성한다.

```
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),

    # keras.layers.Dense(128, activation=tf.nn.relu),
    keras.layers.Reshape(target_shape=(28, 28, 1)),
    keras.layers.Conv2D(filters=32, kernel_size=(3, 3), activation=tf.nn.relu),
    keras.layers.Conv2D(filters=64, kernel_size=(3, 3), activation=tf.nn.relu),
    keras.layers.MaxPooling2D(pool_size=(2, 2)),
    keras.layers.Dropout(0.25),
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(128, activation=tf.nn.relu),
    keras.layers.Dropout(0.5),

    keras.layers.Dense(10)

])
```

■ Keras의 `compile()` 함수를 통해 모델 학습과정 설정하기

- 🔄 모델을 빌드하고 실행하기 전에 컴파일 하는 **훈련 준비 단계**
- 🔄 `keras.Model.compile()`에는 세 개의 파라미터가 중요함

Loss

최적화 과정에서 최소화될 **손실함수**를 설정하는 것

평균 제곱 오차(Mean Squared Error, MSE),
오차 제곱 합(Sum of Squares for Error, SSE)
교차 엔트로피 오차(Cross Entropy Error, CEE)가 자주 사용됨

Optimizer

훈련 과정을 설정하는 것

SGD, Adam 등이 있음

Metrics

훈련을 **모니터링**하기 위해 사용

■ Keras의 compile() 함수를 통해 모델 학습과정 설정하기

🔄 모델학습의 설정 코드는 아래와 같음

```
▶ # create model : 모델을 만든다.  
  
model.compile(optimizer='adam',  
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),  
              metrics=['accuracy'])
```



■ 모델 학습 시 가장 최신의 최고 모델을 저장하는 옵션을 설정함

🔄 콜백함수를 다음과 같이 설정함



```
# 모델 학습시 가장 최신의 최고 모델을 저장하는 인스턴스를 정의
modelCheckpoint = tf.keras.callbacks.ModelCheckpoint('digits_model.h5', save_best_only=True)

# 모델 학습시 loss값을 모니터링하여 조기에 학습을 종료시키는 인스턴스 정의
monitorEarlyStop = tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=10)
```

ModelCheckpoint() 함수

모델이 저장할 때 사용되는 콜백함수

monitor 되고 있는 값을 기준으로 **가장 좋은 값**으로
모델이 “digits_model.h5” 파일 이름으로 **저장**

EarlyStopping() 함수

모델이 더 이상 학습을 못할 경우(Loss, Metric 등의 개선이 없을 경우)
학습 도중 **미리 학습을 종료** 시키는 콜백함수

val_loss가 10번의 epoch을 진행하면서
더 이상 감소되지 않을 경우 학습 **종료**

■ 모델이 저장할 때 사용되는 콜백함수

```
tf.keras.callbacks.ModelCheckpoint(  
    filepath, monitor='val_loss', verbose=0, save_best_only=False,  
    save_weights_only=False, mode='auto', save_freq='epoch', options=None, **kwargs  
)
```

■ 인자 설명

1 filepath

- 1 모델을 저장할 경로를 입력함
- 2 만약 monitor = 'val_loss'일 때,
모델 경로를 '{epoch:02d}-{val_loss:.5f}.h5' 라고 입력하면,
에폭-해당에폭에서의 val_loss.h5로 모델이 저장됨

예 01-0.39121.h5로 저장됨

2 monitor

- 1 모델을 저장할 때, 기준이 되는 값을 지정함(기본값 = 'val_loss').
- 2 validation set의 loss가 가장 작을 때 저장하고 싶으면 'val_loss'를 입력함
- 3 train set의 loss가 가장 작을 때 모델을 저장하고 싶으면 'loss'를 입력함
- 4 외에도 다양한 값들을 기준으로 삼을 수 있음

3 verbose

- 1 0, 1(기본값 = 0)
- 2 1 : 모델이 저장 될 때, '저장되었습니다' 라고 화면에 표시됨
- 3 0 : 화면에 표시되는 것 없이 그냥 바로 모델이 저장됨

4 save_best_only

- 1 True, False(기본값 = False)
- 2 True : monitor 되고 있는 값을 기준으로 가장 좋은 값으로 모델이 저장됨
- 3 False : 매 에폭마다 모델이 filepath{epoch}으로 저장됨
(model0, model1, model2....)



5 save_weights_only

- 1 True, False(기본값 = False)
- 2 True : 모델의 weights만 저장됨
- 3 False : 모델 레이어 및 weights 모두 저장됨

6 mode

- 1 'auto', 'min', 'max'(기본값 = 'auto')
- 2 val_acc : 정확도이기 때문에 클수록 좋으며, max를 입력함
- 3 val_loss : loss 값이기 때문에 값이 작을수록 좋으며, min을 입력함
- 4 auto : 모델이 알아서 min, max를 판단하여 모델을 저장함

7 save_freq

- 1 'epoch' 또는 integer(정수형 숫자)(기본값 = 'epoch')
- 2 epoch : 매 에폭마다 모델이 저장됨
- 3 integer : 숫자만큼의 배치를 진행되면 모델이 저장됨
- 4 숫자 8 입력 시 모델의 저장 : 8번째 배치가 train 된 이후,
16번째 배치가 train 된 이후, ...

8 options

- 1 tf.train.CheckpointOptions를 옵션으로 줄 수 있음
- 2 분산환경에서 다른 디렉토리에 모델을 저장하고 싶을 경우 사용함



- 모델이 더 이상 학습을 못할 경우(loss, metric 등의 개선이 없을 경우) 학습 도중 미리 학습을 종료시키는 콜백함수

```
tf.keras.callbacks.EarlyStopping(  
    monitor='val_loss', min_delta=0, patience=0, verbose=0, mode='auto',  
    baseline=None, restore_best_weights=False  
)
```

인자 설명

1 monitor

- 1 EarlyStopping의 기준이 되는 값을 입력함(기본값 = 'val_loss')
- 2 'val_loss'를 입력하면 val_loss가 더이상 감소되지 않을 경우 EarlyStopping을 적용함
- 3 이외에도 다양한 값을 입력할 수 있음



2 min_delta

- 1 개선된 것으로 간주하기 위한 최소한의 변화량
- 2 min_delta가 0.01이고, 30에폭의 정확도가 0.8532라고 할 때, 만약 31에폭에 정확도가 0.8537라고 하면 이는 0.005의 개선이 있었지만 min_delta값 0.01에는 미치지 못했으므로 개선된 것으로 보지 않음

3 patience

- 1 training이 진행됨에도 더 이상 monitor되는 값의 개선이 없을 경우, 최적의 monitor값을 기준으로 몇 번의 epoch을 진행할 지 정하는 값(기본값 = 0)
- 2 patience는 3이고, 30에폭의 정확도가 99%였을 때, 만약 31번째에 정확도 98%, 32번째에 98.5%, 33번째에 98%라면 더 이상 training을 진행하지 않고 종료시킴



4 verbose

- 1 0 또는 1(기본값 = 0)
- 2 1 : EarlyStopping이 적용될 때, 화면에 적용되었다고 나타냄
- 3 0 : 화면에 나타냄 없이 종료함



5 mode

- 1 'auto', 'min', 'max'(기본값 = 'auto')
- 2 monitor되는 값이 최소가 되어야 하는지, 최대가 되어야 하는지 알려주는 인자
- 3 monitor하는 값이 val_acc(정확도)일 경우, 값이 클수록 좋기 때문에 "max" 입력
- 4 monitor하는 값이 val_loss일 경우, 작을수록 좋기 때문에 "min" 입력
- 5 "auto"는 모델이 알아서 판단함

6 baseline

- 1 모델이 달성해야 하는 최소한의 기준값 선정(기본값 = None)
- 2 patience 이내에 모델이 baseline보다 개선됨이 보이지 않으면 training 중단
- 3 patience가 30이고 baseline이 정확도기준 0.98이라면,
3번의 training 안에 0.98의 정확도를 달성하지 못하면 training이 종료됨

7 restore_best_weights

- 1 True, False(기본값 = False)
- 2 True : training이 끝난 후, model의 weight를 monitor하고 있던 값이
가장 좋았을 때의 weight로 복원
- 3 False : 마지막 training이 끝난 후의 weight로 둬

모델 학습



모델을 학습시키는 것



■ fit() 함수는 epoch 수 만큼 모델을 학습시킴

🔄 설정한 하이퍼파라미터와 콜백함수의 설정 옵션으로 모델을 학습시킴

```
1 # model training : 모델을 학습시킨다.
2
3 history = model.fit(train_images, train_labels,
4                     validation_data=(test_images, test_labels),
5                     epochs=epochs,
6                     batch_size=batch_size,
7                     callbacks=[monitorEarlyStop, modelCheckpoint])
```

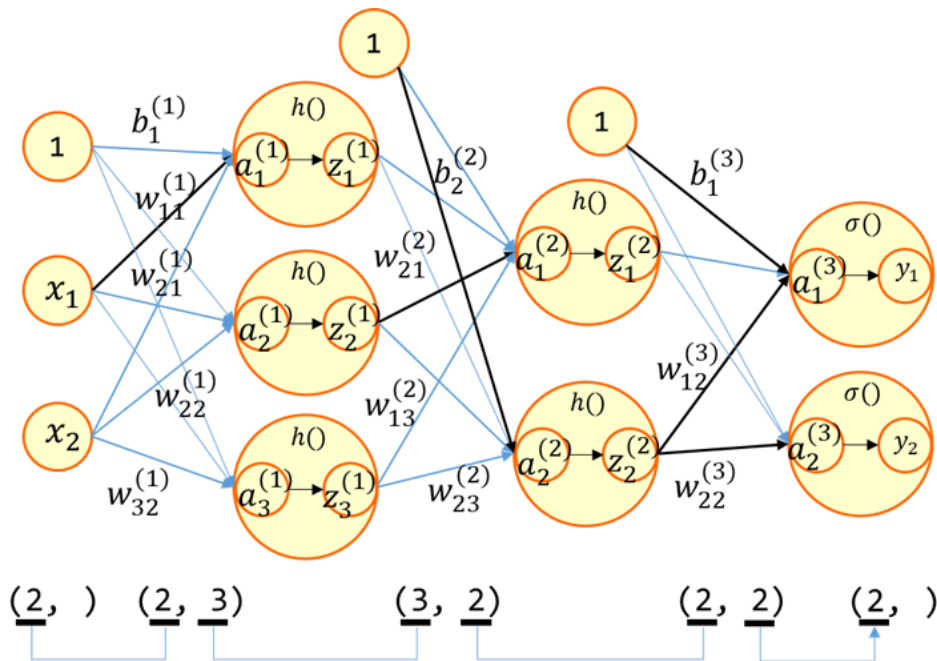
```
Epoch 1/100
469/469 [=====] - 13s 10ms/step - loss: 0.2406 - accuracy: 0.9254 - val_loss: 0.0490 - val_accuracy: 0.9845
Epoch 2/100
469/469 [=====] - 4s 9ms/step - loss: 0.0874 - accuracy: 0.9738 - val_loss: 0.0344 - val_accuracy: 0.9875
Epoch 3/100
469/469 [=====] - 4s 9ms/step - loss: 0.0649 - accuracy: 0.9804 - val_loss: 0.0316 - val_accuracy: 0.9901
Epoch 4/100
469/469 [=====] - 4s 9ms/step - loss: 0.0534 - accuracy: 0.9833 - val_loss: 0.0284 - val_accuracy: 0.9913
Epoch 5/100
469/469 [=====] - 4s 9ms/step - loss: 0.0447 - accuracy: 0.9861 - val_loss: 0.0284 - val_accuracy: 0.9908
Epoch 6/100
469/469 [=====] - 4s 9ms/step - loss: 0.0385 - accuracy: 0.9878 - val_loss: 0.0278 - val_accuracy: 0.9920
Epoch 7/100
469/469 [=====] - 5s 10ms/step - loss: 0.0341 - accuracy: 0.9891 - val_loss: 0.0272 - val_accuracy: 0.9913
Epoch 8/100
469/469 [=====] - 4s 9ms/step - loss: 0.0314 - accuracy: 0.9901 - val_loss: 0.0303 - val_accuracy: 0.9909
Epoch 9/100
469/469 [=====] - 4s 9ms/step - loss: 0.0284 - accuracy: 0.9906 - val_loss: 0.0281 - val_accuracy: 0.9918
Epoch 10/100
469/469 [=====] - 4s 9ms/step - loss: 0.0274 - accuracy: 0.9913 - val_loss: 0.0264 - val_accuracy: 0.9928
```

모델학습이 완료됨

VM에 “digits_model.h5”란 이름으로 학습된 모델이 저장됨

```
Epoch 1/100
469/469 [=====] - 16s 10ms/step - loss: 0.2316 - accuracy: 0.9301 - val_loss: 0.0568 - val_accuracy: 0.9829
Epoch 2/100
469/469 [=====] - 5s 11ms/step - loss: 0.0857 - accuracy: 0.9747 - val_loss: 0.0370 - val_accuracy: 0.9874
Epoch 3/100
469/469 [=====] - 4s 9ms/step - loss: 0.0643 - accuracy: 0.9804 - val_loss: 0.0335 - val_accuracy: 0.9892
Epoch 4/100
469/469 [=====] - 4s 9ms/step - loss: 0.0506 - accuracy: 0.9844 - val_loss: 0.0314 - val_accuracy: 0.9885
Epoch 5/100
469/469 [=====] - 4s 9ms/step - loss: 0.0438 - accuracy: 0.9863 - val_loss: 0.0314 - val_accuracy: 0.9901
Epoch 6/100
469/469 [=====] - 4s 9ms/step - loss: 0.0368 - accuracy: 0.9883 - val_loss: 0.0267 - val_accuracy: 0.9915
Epoch 7/100
469/469 [=====] - 4s 9ms/step - loss: 0.0341 - accuracy: 0.9897 - val_loss: 0.0270 - val_accuracy: 0.9918
Epoch 8/100
469/469 [=====] - 4s 9ms/step - loss: 0.0304 - accuracy: 0.9900 - val_loss: 0.0244 - val_accuracy: 0.9926
Epoch 9/100
469/469 [=====] - 4s 9ms/step - loss: 0.0267 - accuracy: 0.9911 - val_loss: 0.0245 - val_accuracy: 0.9926
Epoch 10/100
469/469 [=====] - 4s 9ms/step - loss: 0.0238 - accuracy: 0.9924 - val_loss: 0.0297 - val_accuracy: 0.9913
Epoch 11/100
469/469 [=====] - 4s 9ms/step - loss: 0.0215 - accuracy: 0.9926 - val_loss: 0.0249 - val_accuracy: 0.9930
Epoch 12/100
469/469 [=====] - 4s 9ms/step - loss: 0.0220 - accuracy: 0.9928 - val_loss: 0.0301 - val_accuracy: 0.9920
Epoch 13/100
469/469 [=====] - 4s 9ms/step - loss: 0.0189 - accuracy: 0.9934 - val_loss: 0.0273 - val_accuracy: 0.9917
Epoch 14/100
469/469 [=====] - 4s 9ms/step - loss: 0.0181 - accuracy: 0.9942 - val_loss: 0.0255 - val_accuracy: 0.9933
Epoch 15/100
469/469 [=====] - 4s 9ms/step - loss: 0.0167 - accuracy: 0.9949 - val_loss: 0.0260 - val_accuracy: 0.9929
Epoch 16/100
469/469 [=====] - 4s 9ms/step - loss: 0.0181 - accuracy: 0.9941 - val_loss: 0.0308 - val_accuracy: 0.9922
Epoch 17/100
469/469 [=====] - 4s 9ms/step - loss: 0.0143 - accuracy: 0.9952 - val_loss: 0.0284 - val_accuracy: 0.9934
Epoch 18/100
469/469 [=====] - 4s 9ms/step - loss: 0.0143 - accuracy: 0.9954 - val_loss: 0.0275 - val_accuracy: 0.9924
```

3층 인공지능망 모델 입력층, 은닉층, 출력층으로 신호 전달하는 과정



$(2, 3)$

$$\begin{pmatrix} w_{11}^{(1)} & w_{21}^{(1)} & w_{31}^{(1)} \\ w_{12}^{(1)} & w_{22}^{(1)} & w_{32}^{(1)} \end{pmatrix}$$

$$(b_1^{(1)}, b_2^{(1)}, b_3^{(1)})$$

$(3, 2)$

$$\begin{pmatrix} w_{11}^{(2)} & w_{21}^{(2)} \\ w_{12}^{(2)} & w_{22}^{(2)} \\ w_{13}^{(2)} & w_{23}^{(2)} \end{pmatrix}$$

$$(b_1^{(2)}, b_2^{(2)})$$

$(2, 2)$

$$\begin{pmatrix} w_{11}^{(3)} & w_{21}^{(3)} \\ w_{12}^{(3)} & w_{22}^{(3)} \end{pmatrix}$$

$$(b_1^{(3)}, b_2^{(3)})$$

가중치

편향

4층 인공지능망 모델 4층 인공지능망 모델의 계층구조로 이루어진 네트워크 구조



모델 학습 시 history 변수에 저장된
loss value, accuracy 정보를 그래프로 시각화 해보자



모델 학습시 history 객체 저장해 두었던 loss Value, accuracy 정보를 그래프로 표현해 봅니다.

```
import matplotlib.pyplot as plt
```

```
fig, loss_ax = plt.subplots()
```

```
fig, acc_ax = plt.subplots()
```

```
loss_ax.plot(history.history['loss'], 'ro', label='train loss') # train loss
```

```
loss_ax.plot(history.history['val_loss'], 'r:', label='val loss') # validation loss
```

```
loss_ax.set_xlabel('epoch') # x축 label은 epoch
```

```
loss_ax.set_ylabel('loss') # y축 label은 loss
```

```
loss_ax.legend(loc='upper left') # 범례의 위치는 왼쪽 위
```

```
acc_ax.plot(history.history['accuracy'], 'bo', label='train accuracy') # train accuracy
```

```
acc_ax.plot(history.history['val_accuracy'], 'b:', label='val accuracy') # validation accuracy
```

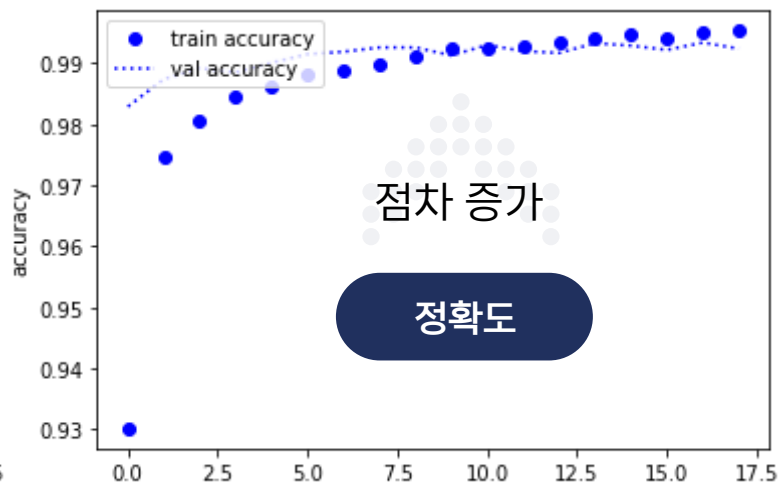
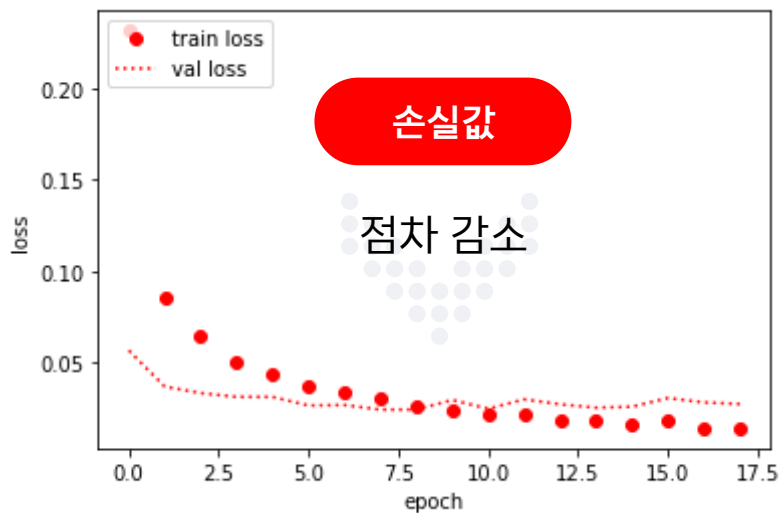
```
acc_ax.set_ylabel('accuracy') # y축 label은 accuracy
```

```
acc_ax.legend(loc='upper left') # 범례의 위치는 왼쪽 위
```

```
plt.show()
```


모델 학습 시 history 변수에 저장된
loss value, accuracy 정보를 그래프로 시각화 해보자

학습과정의 loss value, accuracy 시각화 결과





■ 학습된 모델을 테스트 데이터셋으로 평가함

🔄 테스트 데이터셋으로 평가한 모델의 정확도는 99.24% 임



model evaluate : 모델을 평가해 봅니다.

```
test_loss, test_acc = model.evaluate(test_images, test_labels)
```

```
print('Test accuracy:', test_acc)
```



```
313/313 [=====] - 1s 3ms/step - loss: 0.0275 - accuracy: 0.9924  
Test accuracy: 0.9923999905586243
```

■ 테스트 데이터셋으로 학습된 모델을 통해 숫자(Digit) Label을 예측함

- 예측 결과에서 첫 번째 이미지의 예측 값을 출력함
- 아래 결과에서 10,000장의 형상은 (10000, 10)인 것을 볼 수 있음
- 첫 번째 이미지의 예측결과에서 일곱 번째 인덱스 값이 가장 큰 것을 알 수 있음

```
1 # test dataset을 우리가 만든 모델을 통해 digit label을 예측합니다.
2
3 predictions = model.predict(test_images)
4
5 print(predictions.shape)           # (10000, 10)
6 print(predictions[0,])           # 숫자 7의 위치에서 가장 큰값을 출력
7 # [-4869.5664  12.003499 -978.2258 -1250.3693 -1456.89 -3117.7454 -7133.385  6700.074 -5340.4604  487.48645 ]
8
9 print("예측값의 위치:", np.argmax(predictions[0,])) # 예측값의 위치: 7
```

➡ 모델이 숫자 '7'을 예측함

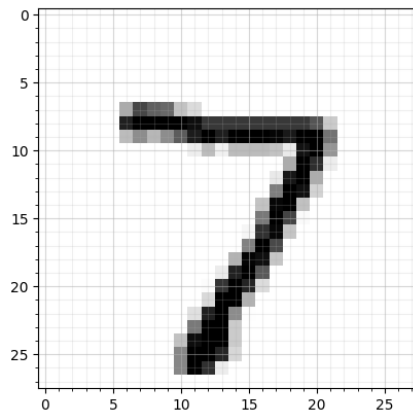
0	1	2	3	4	5	6	7	8	9
-4869.56	12.00	-978.22	-1250.36	-1456.89	-3117.74	-7133.38	6700.07	-5340.46	487.48

가장 큰 값

예측 결과 중에서 0번째 숫자의 이미지와 예측한 레이블(Label)을 출력해 보자

- show_digit_image() 함수로 특정 이미지의 레이블 출력
- 아래 코드와 테스트 이미지에서 첫 번째 이미지의 숫자 레이블이 “7”인 것을 알 수 있음

```
1 # 예측 결과 중에서 특정한 숫자(digit)의 이미지와 예측한 label 을 Display 합니다.
2 # - 첫 번째 인덱스의 예측값 출력
3
4 selected_digit = 0
5
6 result = predictions[selected_digit]
7 result_number = np.argmax(result)
8 print('Number is %2i' % (result_number))
9
10 show_digit_image(test_images[selected_digit]) # 특정 이미지의 레이블 출력
```

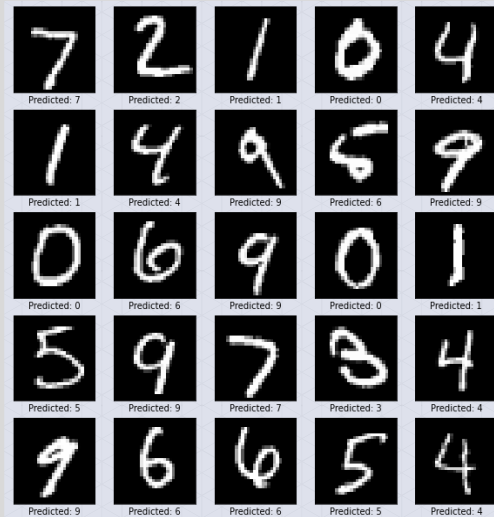




■ show_sample() 함수로 테스트 데이터셋에서 25장의 샘플을 출력함

- 인자 값으로 테스트 데이터, 테스트 데이터의 예측 레이블을 전달함
- 실행 결과, 매우 잘 예측한 것을 알 수 있음

```
1 # show_sample() 함수로 테스트 데이터의 예측한 결과를 Display 합니다.  
2  
3 show_sample(test_images, ['Predicted: %d' % np.argmax(result) for result in predictions])
```

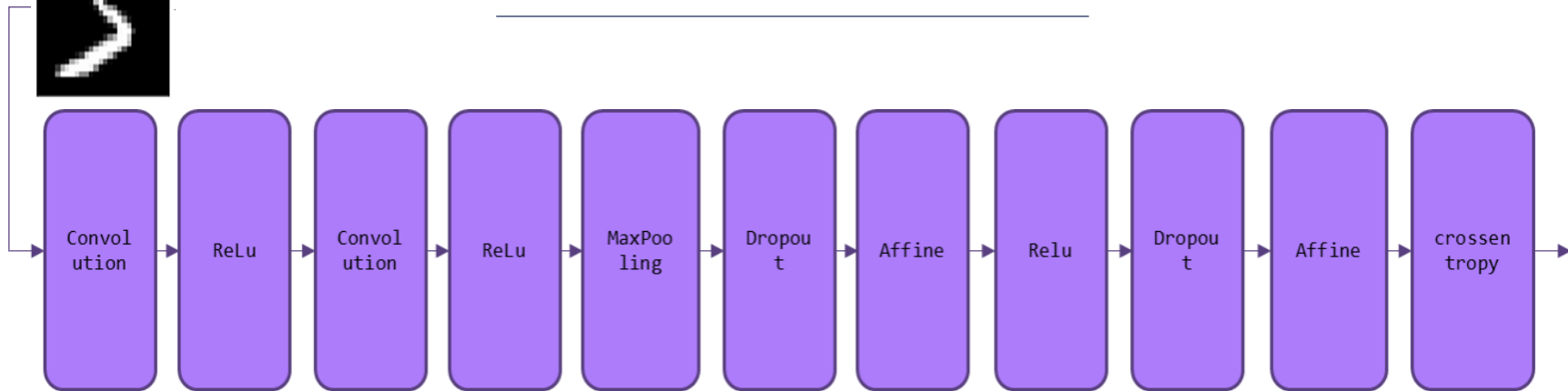


인공신경망 모델 개선의 필요성과 수행방법

MNIST 데이터셋의 정확도를 높이기 위해 모델 개선이 필요하며,
데이터 전처리, 인공신경망 학습의 고급기술을 이용해 모델 개선을 수행함



개별 과제로 모델링 수행하기



인공신경망 네트워크 구성